

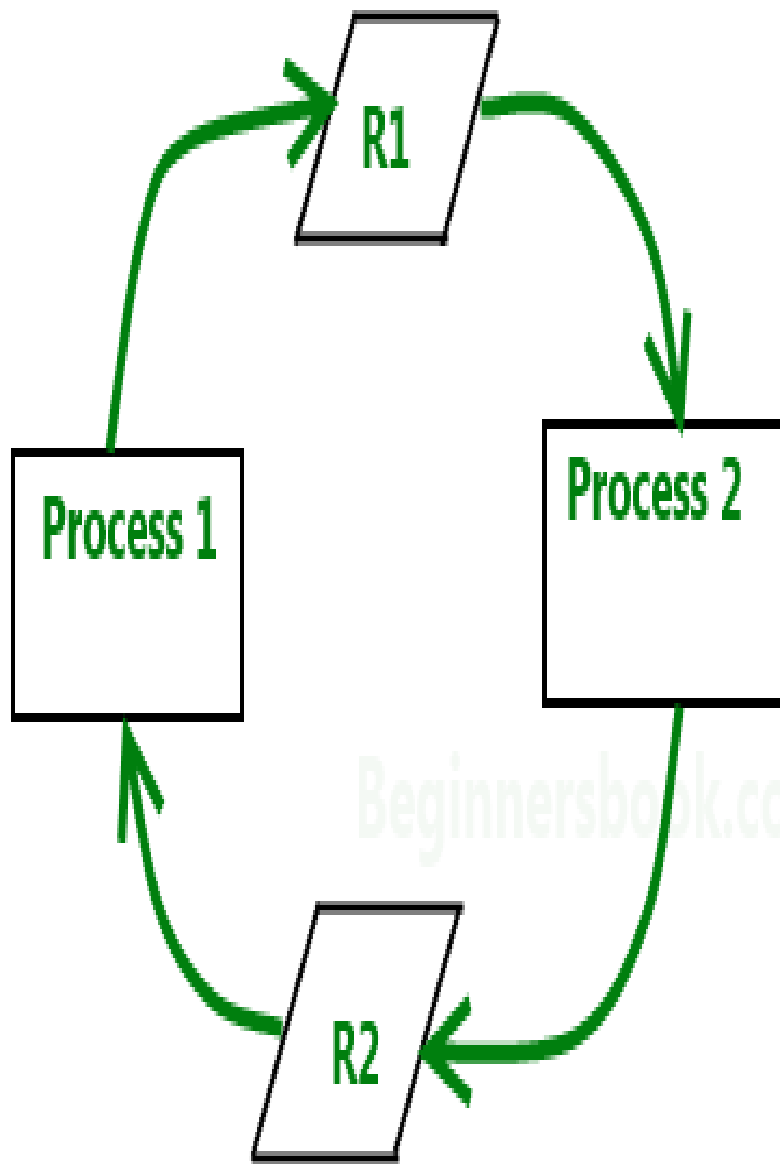
Unit: 4 Process Deadlocks

Presented by
Er. Binod Kumar Rajbhar
BE-Computer
ME-Information System Engineering

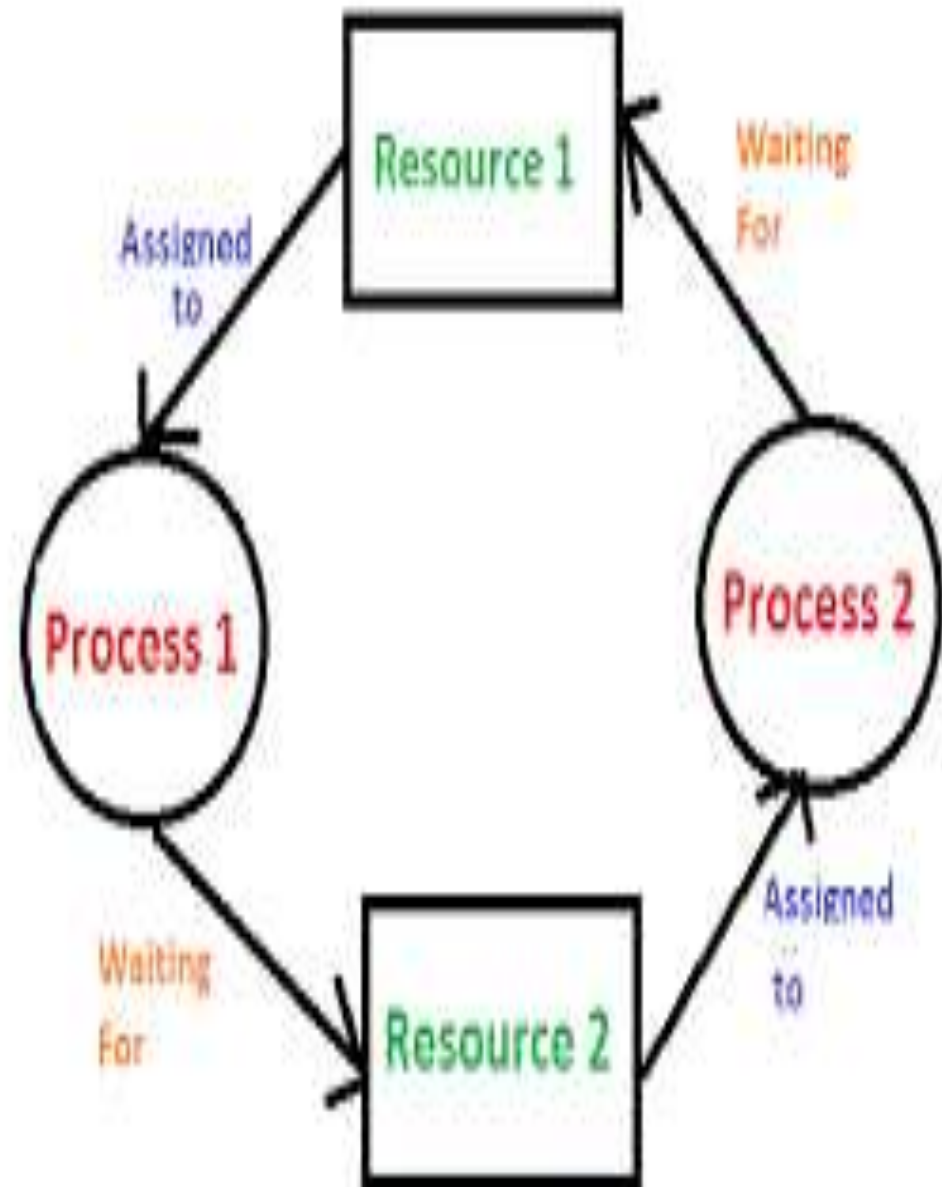
Introduction

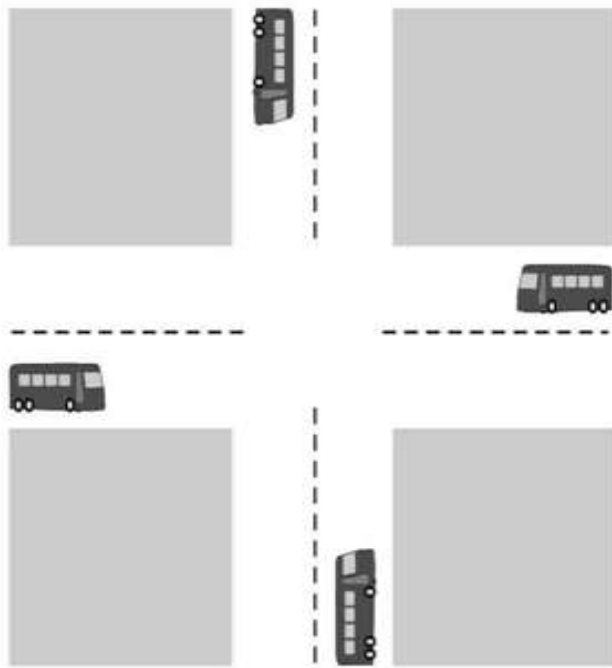
Definition:

- ✓ If two or more process are waiting on happening of some events, which never happens then we say these process are involved in deadlock then that state is called deadlock.
- ✓ A set of process is in a deadlock state if each process in the set is waiting for an event that caused by only another process in the set.
- ✓ Deadlock is a situation in which two computer programs sharing the same resource are effectively preventing each other from accessing the resource, and none of them can be awakened.

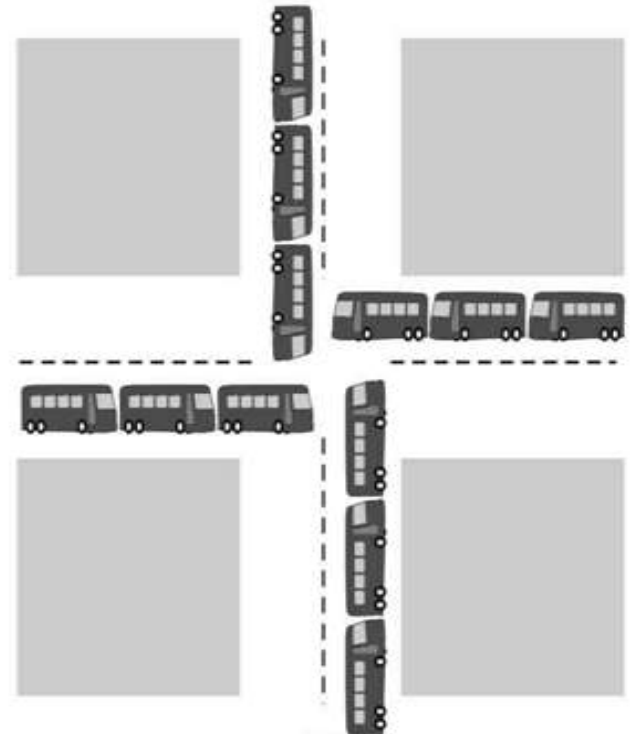


Process P1 holds resource R2 and requires R1 while
Process P2 holds resource R1 and requires R2.





(a)



(b)

Resources

- ❖ Resources are the passive entities needed by processes to do their work.
- ❖ A resource can be:
 - ✓ Hardware device (e.g. A tape drive)
 - ✓ Piece of information (e.g. A locked record in database)
- ❖ Example of resources include CPU time, disk space, memory etc.

There are two types of resources:

-

Preemptable resource:

- ❖ That can be taken away from the process owing it with no ill effects.
- ❖ Example: memory

Non-preemptable resource:

- ❖ That can not be taken away from its current owner without causing the computation to fail.
- ❖ Example: CD recorders

The sequence of events required to use a resource is:

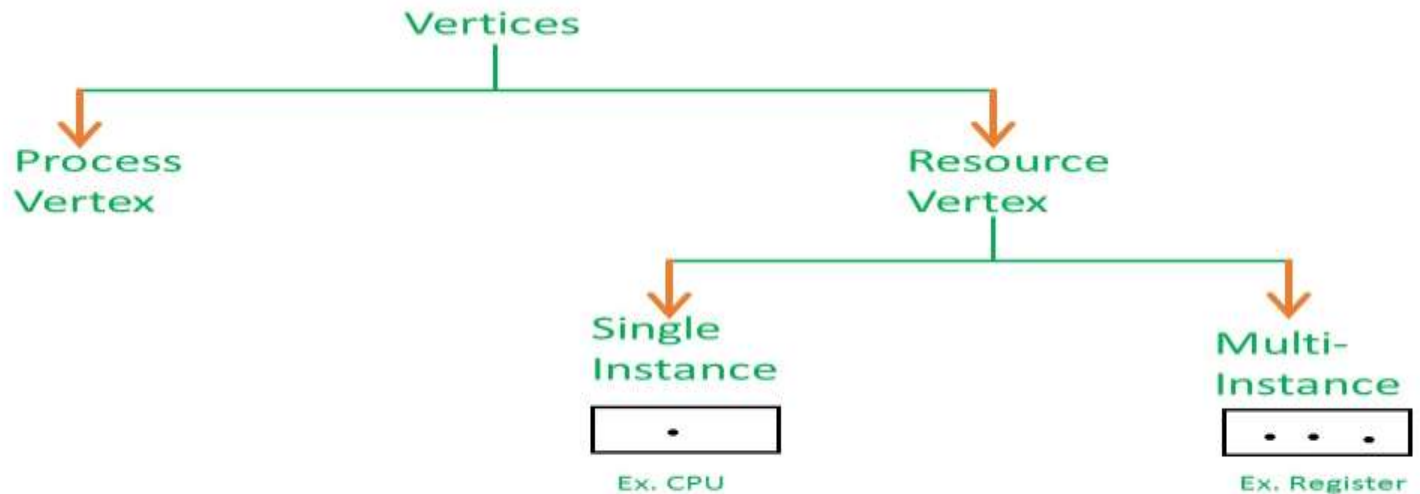
1. Request the resource
1. Use the resource
1. Release the resource

Resource Allocation Graph(RAG)

- ✓ Resource allocation graph is explained as what is the state of the system in terms of processes and resources.
- ✓ Like how many resources are available , how many are allocated and what is the request of each process.
- ✓ Everything can be represented in terms of diagram.
- ✓ One of the advantage of having a diagram is, sometimes it is possible to see a deadlock directly by using RAG which is not possible by looking the table.

- ✓ Table are better if the system contains lots of process and resource and graph is better if the system contains less number of process and resources.
- ✓ We know that any graph contains vertices and edges, so RAG also contains vertices and edges.
- ✓ **In RAG vertices are two types:**
 1. **Process vertex:** Every process will be represented as a process vertex. Generally , the process will be represented with a circle.
 2. **Resource vertex:** Every resource will be represented as resource vertex. It is also of two types:

- I. **Single instance types resources:** it represents as a box, inside the box, there will be one dot. So the number of dots indicates how many instances are present of each resource types.
- II. **Multi-resource instance types resource:** it also represents as a box, inside the box, there will be many dots presents.

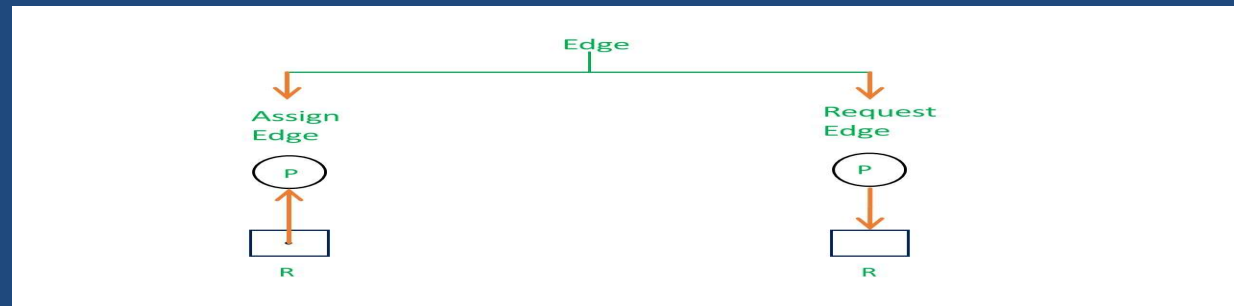


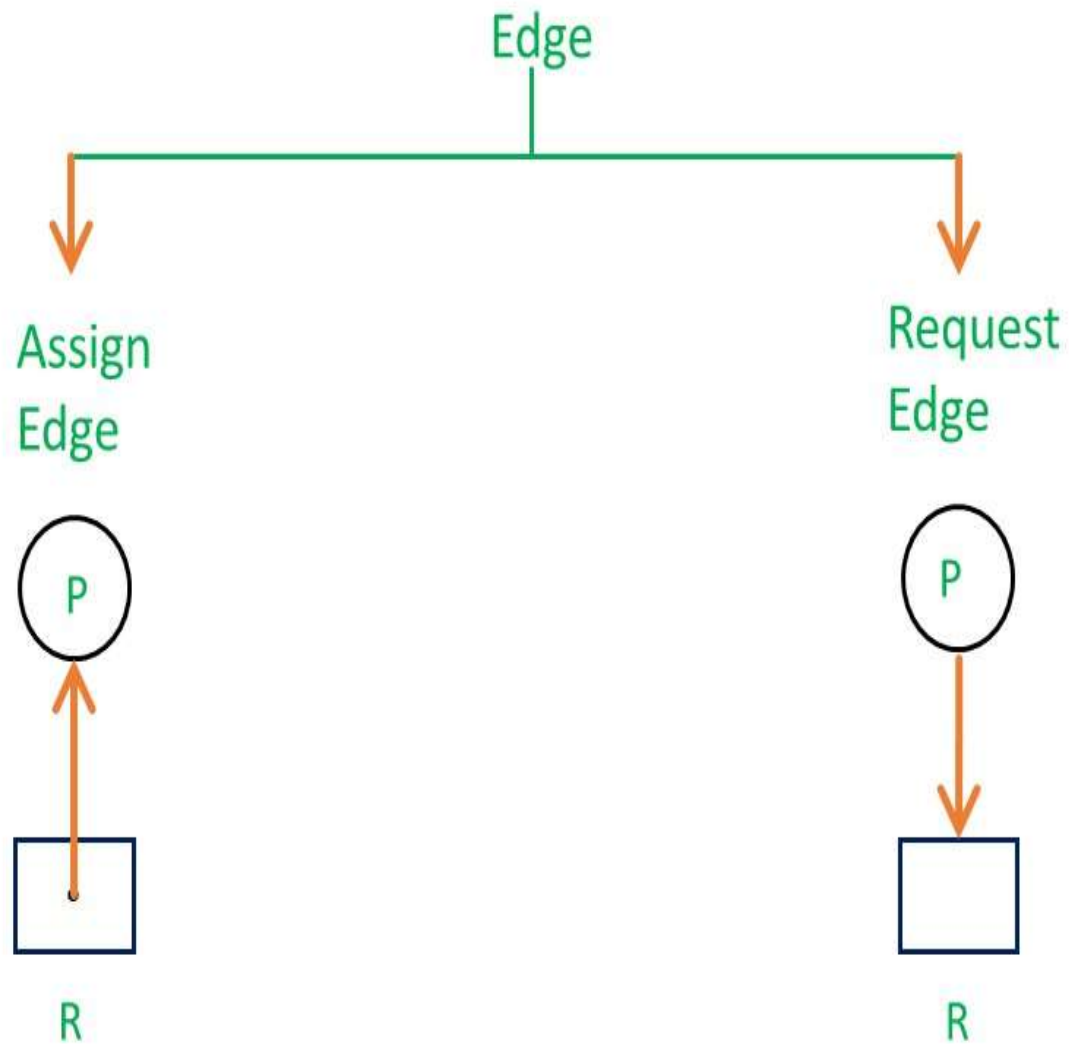
There are two types of edges in RAG:

1) **Assign Edge**: if you already assign a resource to a process then it is called Assign Edge.

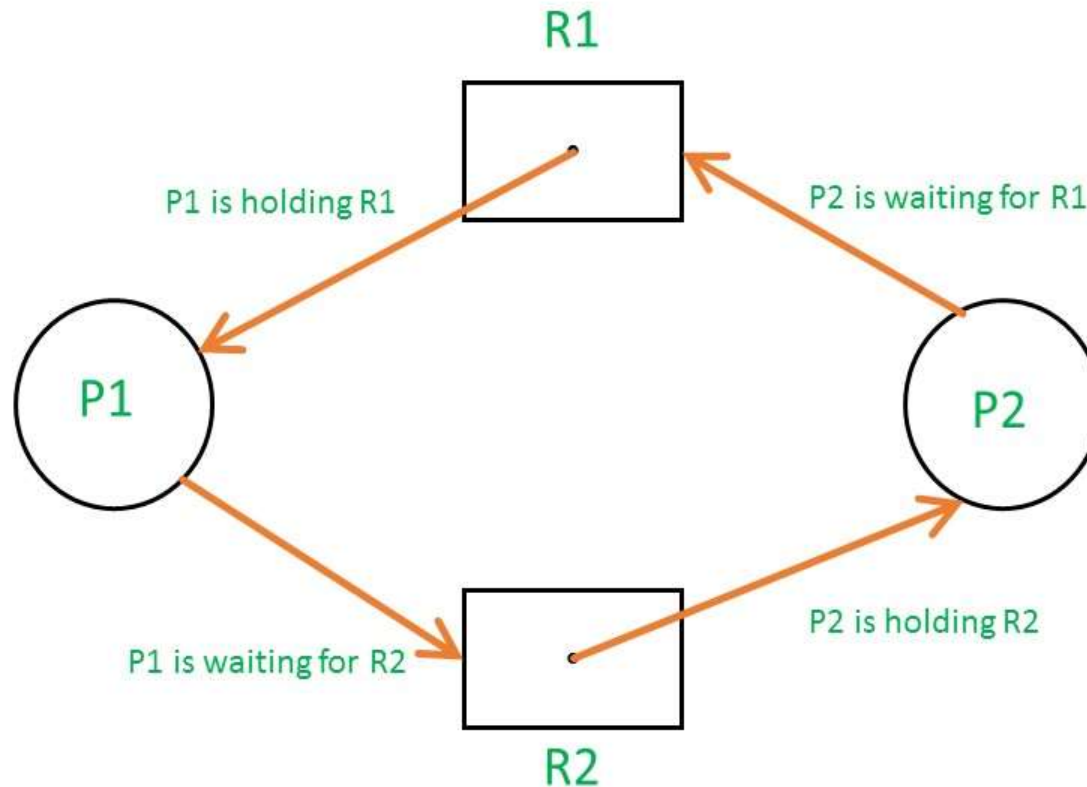
2. **Request Edge**: it means in future the process might want some resource to complete the execution that is called request edge.

- If a process is using a resource, an arrow is drawn from the resource node to the process node. If a process is requesting a resource, an arrow from the node to the resource node.





Single Instances RAG

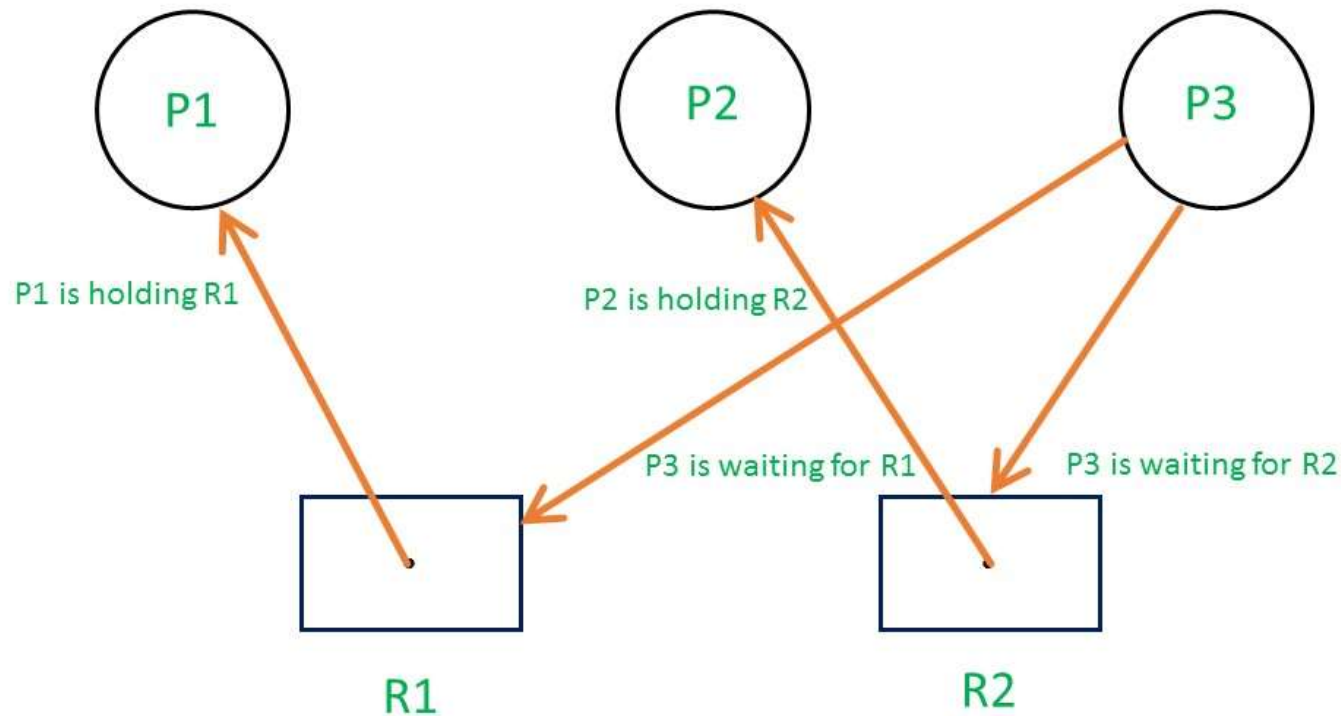


SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK

Single Instances Resource types with Deadlock

- ✓ If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be deadlock.
- ✓ For examples: if process P1 holds resource R1, process P2 holds resources R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then the process P1 and process P2 will be in deadlock.

Single Instances Resource types without Deadlock

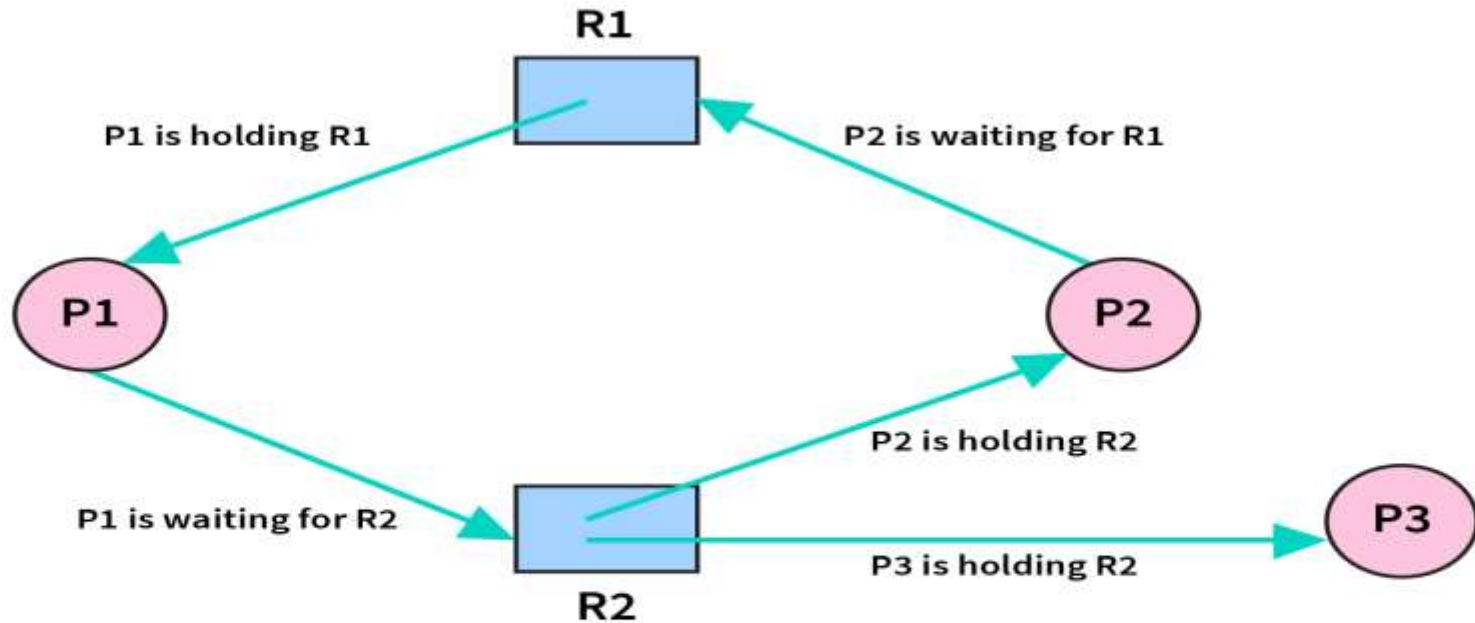


SINGLE INSTANCE RESOURCE TYPE WITHOUT DEADLOCK

Single Instances Resource types without Deadlock

- ✓ Here is another examples that shows processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources.
- ✓ In this examples, there is no deadlock because there is no circular dependency. So cycle in single instance resource types is the in sufficient condition for deadlock

Multi-Instances RAG



Multi Instances Without Deadlock

SCALER
Topics

Multi-Instances RAG without deadlock

- ✓ From the above examples, it is not possible to say the RAG is in a safe state or in an unsafe state, so to see the state of this RAG, construct the allocation matrix and request matrix:

Process	AllocationResources			Request Resources	
	R1	R2		R1	R2
P1	1	0		0	1
P2	0	1		1	0
P3	0	1		0	0

✓ Total number of process are three: P1, P2 & P3 and total number of resources are two: R1 & R2.

✓ Allocation matrix:

- For constructing the allocation matrix, just to go the resources and see to which process it is allocated.
- R1 is allocated to P1, therefore write 1 in allocation matrix and similarly, R2 is allocated to P2 as well as P3 and for the remaining elements just write 0

Request matrix:

- In order to find out the request matrix, you have to go to the process and see the outgoing edges.
- P1 is requesting resource R2, so write 1 in the matrix and similarly, P2 is requesting R1 and for the remaining element write 0.
- So now available resource is = (0,0)

Necessary conditions for Deadlocks

❖ **Mutual Exclusion:**

- ✓ A resource can only be shared in mutually exclusive manner.
- ✓ It implies, if two processes cannot use the same resource at the same time.

❖ **Hold and Wait:**

- ✓ A process waits for some resources while holding another resource at the same time.

❖ **No preemption:**

- ✓ The process which once scheduled will be executed till the completion. No other process can be scheduled by the scheduler meanwhile.

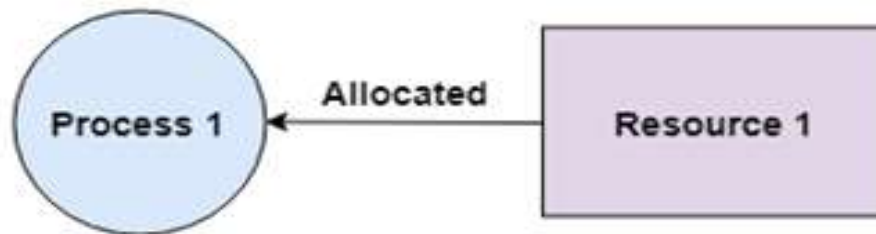
❖ **Circular Wait:**

- ✓ All the processes must be waiting for the resources in a cyclic manner so that the last process is waiting for the resource which is being held by the first process.

Necessary conditions for Deadlocks

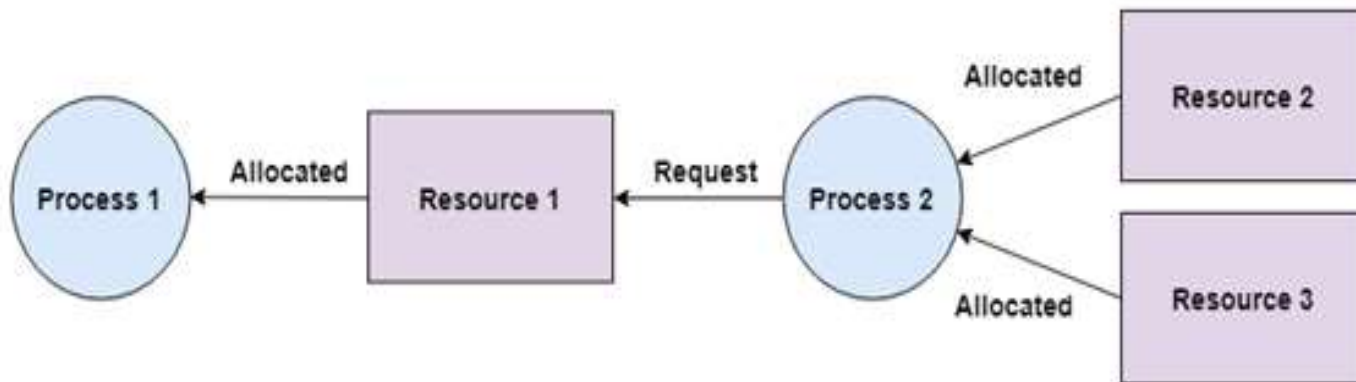
❖ Mutual Exclusion:

- ✓ There should be a resource that can only be held by one process at a time.
- ✓ In the diagram below, there is a single instance of Resource 1 and it is held by Process 1 only.



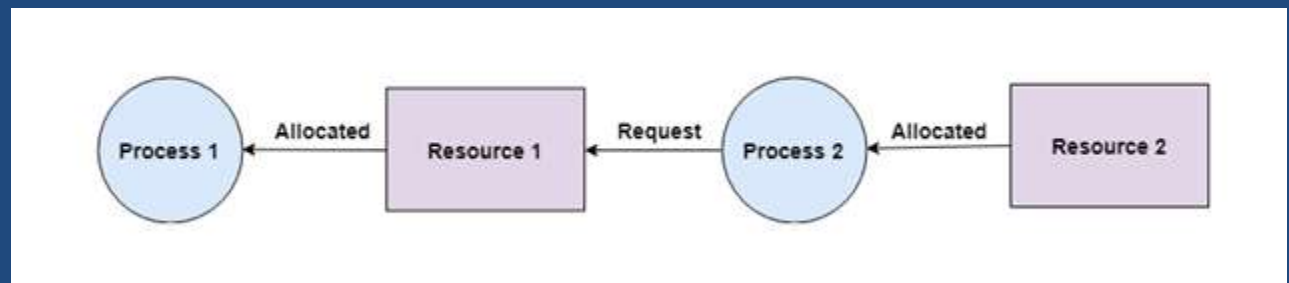
Hold and Wait

- ❖ A process can hold multiple resources and still request more resources from other processes which are holding them.
- ❖ In the diagram given below, Process 2 holds Resource 2 and Resource 3 and is requesting the Resource 1 which is held by Process 1.



No Preemption

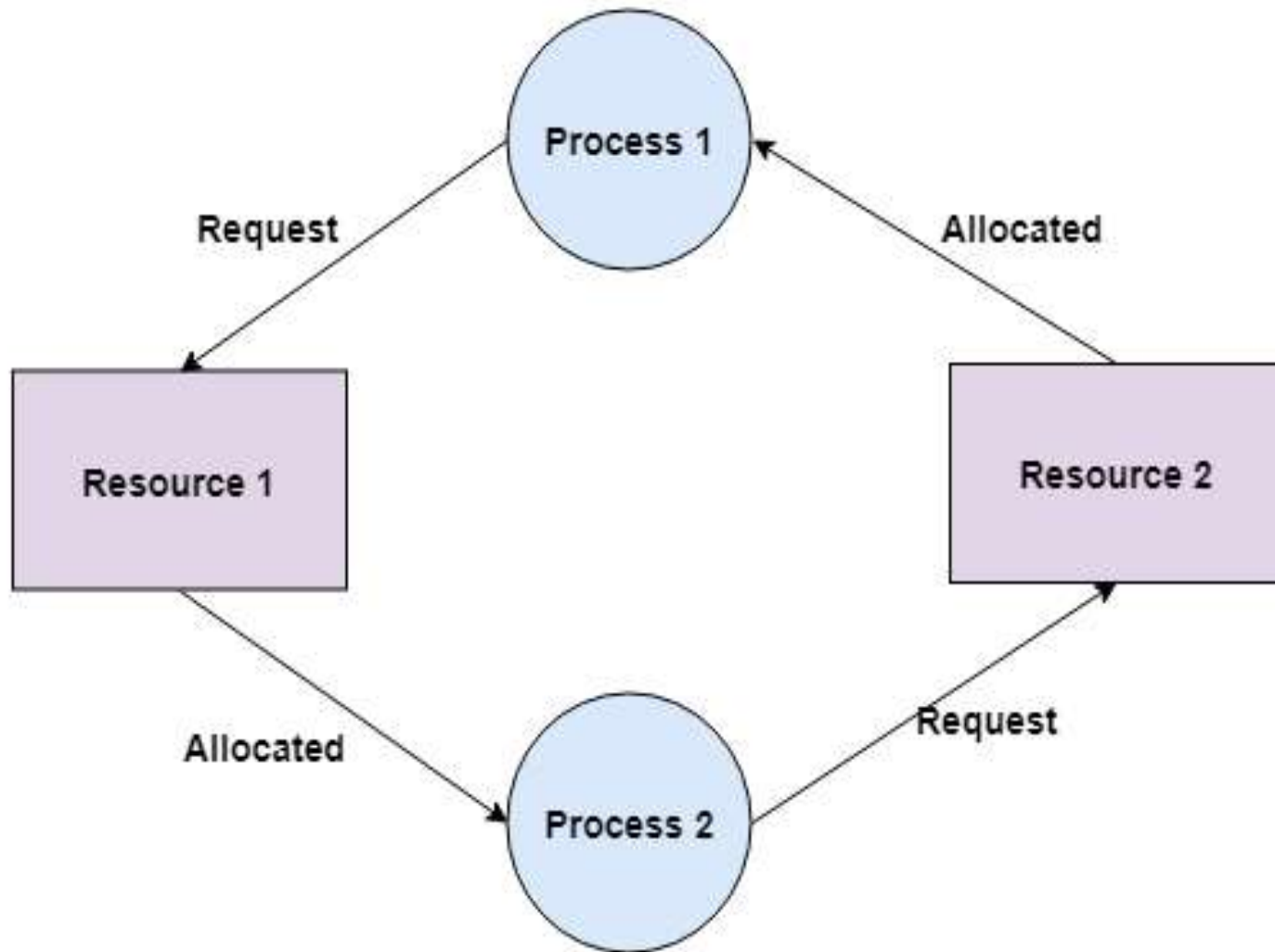
- ✓ Resource already allocated to a process cannot be preempted, resources cannot be removed from the processes are used to completion or released voluntarily by the process holding it.
- ✓ In the diagram below, Process 2 cannot preempt Resource 1 from Process 1.
- ✓ It will only be released when Process 1 relinquishes it voluntarily after its execution is complete.



Circular Wait

- ✓ A process is waiting for the resource held by the second process, which is waiting for the resource held by the third process and so on, till the last process is waiting for a resource held by the first process.
- ✓ This forms a circular chain.
- ✓ For example: Process 1 is allocated Resource 2 and it is requesting Resource 1.
- ✓ Similarly, Process 2 is allocated Resource 1 and it is requesting Resource 2.
- ✓ This forms a circular wait loop.

Circular Wait



Deadlock Handling

- Deadlock prevention
- Deadlock avoidance
- Deadlock detection
- Recovery from deadlock

Deadlock Ignorance (Ostrich Algorithms)

- ❖ Deadlock Ignorance is the most widely used approach among all the mechanism.
- ❖ This is being used by many operating systems mainly for end user uses.
- ❖ In this approach, the Operating system assumes that deadlock never occurs.
- ❖ It simply ignores deadlock. This approach is best suitable for a single end user system where User uses the system only for browsing and all other normal stuff.
- ❖ There is always a tradeoff between Correctness and performance.
- ❖ The operating systems like Windows and Linux mainly focus upon performance.
- ❖ However, the performance of the system decreases if it uses deadlock handling mechanism all the time if deadlock happens 1 out of 100 times then it is completely unnecessary to use the deadlock handling mechanism all the time.
- ❖ In these types of systems, the user has to simply restart the computer in the case of deadlock. Windows and Linux are mainly using this approach.

Deadlock prevention:

- ❖ Deadlock happens only when Mutual Exclusion, hold and wait, No preemption and circular wait holds simultaneously.
- ❖ If it is possible to violate one of the four conditions at any time then the deadlock can never occur in the system.
- ❖ The idea behind the approach is very simple that we have to fail one of the four conditions but there can be a big argument on its physical implementation in the system.

Eliminate Mutual Exclusion

- ✓ It is not possible to dis-satisfy the mutual exclusion because some resources, such as the tape drive and printer, are inherently non-shareable.
- ✓ Mutual exclusion means that unshareable resources cannot be accessed simultaneously by processes.
- ✓ Shared resources do not cause deadlock but some resources can't be shared among processes, leading to a deadlock.
- ✓ **For example:** *read* operation on a file can be done simultaneously by multiple processes, but *write* operation cannot. *Write* operation requires sequential access, so, some processes have to wait while another process is doing a *write* operation.

Eliminate Hold and wait

1. Allocate all required resources to the process before the start of its execution, this way hold and wait condition is eliminated but it will lead to low device utilization. for example, if a process requires printer at a later time and we have allocated printer before the start of its execution printer will remain blocked till it has completed its execution.
2. The process will make a new request for resources after releasing the current set of resources. This solution may lead to starvation.

Eliminate No Preemption:

Preempt resources from the process when resources required by other high priority processes.

Eliminate Circular Wait :

Each resource will be assigned with a numerical number. A process can request the resources increasing/decreasing. order of numbering.

For Example, if P1 process is allocated R5 resources, now next time if P1 ask for R4, R3 lesser than R5 such request will not be granted, only request for resources more than R5 will be granted.

Deadlock Avoidance

- ✓ Deadlock avoidance can be done with Banker's Algorithm.

Banker's Algorithm:

Banker's Algorithm is resource allocation and deadlock avoidance algorithm which test all the request made by processes for resources, it checks for the safe state, if after granting request system remains in the safe state it allows the request and if there is no safe state it doesn't allow the request made by the process.

Inputs to Banker's Algorithm:

1. Max need of resources by each process.
2. Currently, allocated resources by each process.
3. Max free available resources in the system.

The request will only be granted under the below condition:

1. If the request made by the process is less than equal to max need to that process.
2. If the request made by the process is less than equal to the freely available resource in the system.

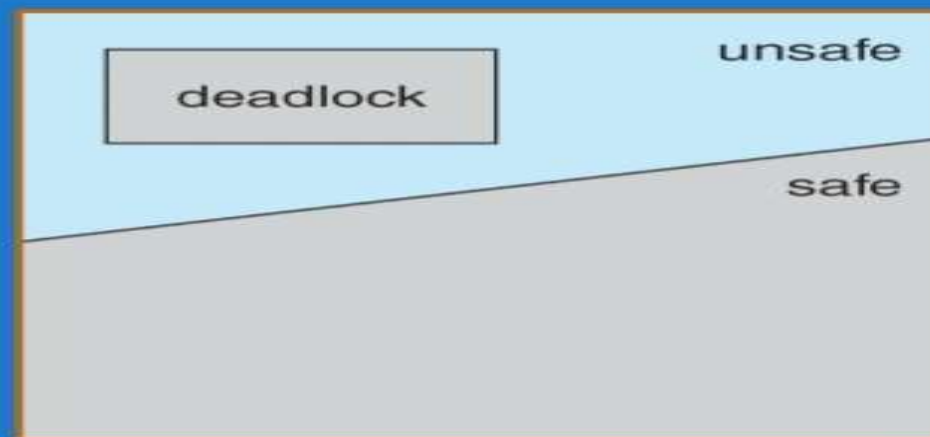
Problem with Banker's Algorithm

- ✓ An algorithm requires fixed number of resources, some processes dynamically change the number of resources.
- ✓ An algorithm requires the number of resources in advanced, it is very difficult to predict the resources in advanced.
- ✓ Algorithms predict all process returns within finite time, but the system does not guarantee it.

Safe and Unsafe States

Basic Facts

- If a system is in safe state, then no deadlocks.
- If a system is in unsafe state, then possibility of deadlock.
- Avoidance means ensuring that a system will never enter an unsafe state.



Deadlock Recovery

- ✓ Traditional operating system such as windows don't deal recovery as it is time and space consuming process.
- ✓ Real time operating systems use deadlock recovery. Some of the common recovery methods are:

For Resource:

- ❖ **Preempt the resource:** we can snatch one of the resources from the owner of the resources (process) and give it to the other process with the expectation that it will complete the execution and release this resource sooner.
- ❖ **Rollback to a safe state:** System passes through various states to get into the deadlock state. Operating system can roll back the system to the previous safe states. For this purpose, OS needs to implement check pointing at every state. This movement, we get into deadlock, we rollback all the allocations to get into the previous safe state.

For Process:

- ✓ **Kill a process:** Killing a process can solve our problem but the bigger concern is to decide which process to kill. Generally , operating system kills a process which has done least amount of work until now.
- **Kill all Process:** This is not a suggestible approach but can be implemented if the problem becomes very serious. Killing all process will lead to inefficiency in the system become all the processes will execute again from starting.

Difference between Starvation and Deadlock

Deadlock

The deadlock situation occurs when one of the processes got blocked.

Deadlock is an infinite process.

Every Deadlock always has starvation.

Deadlock happens then Mutual exclusion, hold and wait. Here, preemption and circular wait do not occur simultaneously.

Starvation

Starvation is a situation where all the low priority processes got blocked, and the high priority processes execute.

Starvation is a long waiting but not an infinite process.

Every starvation does n't necessarily have a deadlock.

It happens due to uncontrolled priority and resource management.